TCP Congestion Control: AIMD

The last part of TCP that we're going to study is congestion control.

We'll cover the following

- Requirements of a Congestion Control Algorithm
- TCP Congestion Control Algorithms
  - Additive Increase Multiplicative Decrease
- Quick Quiz!

We've already looked at what congestion control is in a previous lesson. Let's get into some TCP-specific congestion control algorithms now!

But first:

Requirements of a Congestion Control Algorithm

1. The congestion control scheme **must avoid congestion**. In practice, this means that the algorithm should ensure that the bandwidth allocated to a certain host at any given time does not exceed the bandwidth of the bottleneck link.
2. The congestion control scheme **must be efficient**. The congestion control scheme should ensure that the available bandwidth is efficiently used. Namely that the bandwidth allocated to a certain host at any given time doesn't fall too far below the bandwidth of the bottleneck link.
3. The congestion control scheme **should be fair**. Most congestion schemes aim at achieving max-min fairness, which we had a lesson dedicated to.

TCP Congestion Control Algorithms

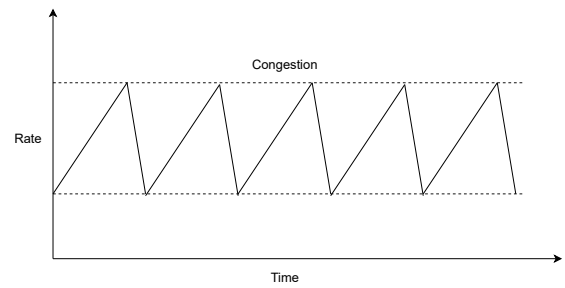
To implement most TCP congestion control algorithms, a TCP host must be able to control its **transmission rate**. In order to do so, it can constrain its **sending window**. Recall that after transmitting data equal to the window size, the sender must pause and wait at least one RTT for the *ACK* before it can transmit more data. Thus the maximum data rate is: $\frac{window}{rtt}$ where $window$ is the maximum between the host's sending window and the window advertised by the receiver.

Hence, TCP's congestion control scheme can restrict sending windows based on a **congestion window**. The current value of the congestion window is stored in the TCB of each TCP connection. The value of the window that can be used by the sender is $min(congestion\ window,\ receiving\ window,\ sending\ window)$.

Additive Increase Multiplicative Decrease

The **Additive Increase Multiplicative Decrease** algorithm *decreases* the transmission rate of a host when congestion has been detected in the network. Congestion is detected when acknowledgments for packets do not arrive before the transmission timer times out. Furthermore, it *increases* their transmission rate when the network is *not* congested. Hence, the rate allocated to each host fluctuates with time, depending on the feedback received from the network.

The figure below illustrates the evolution of the transmission rates allocated to a host in a network.



How the transmission rate of a host changes with AIMD. Notice the 'saw tooth' pattern.

The **Additive Increase** part of the TCP congestion control increments the congestion window by *MSS* bytes every round-trip time. In the TCP literature, this phase is often called the *congestion avoidance* phase. Once congestion is detected, the **Multiplicative Decrease** part of the TCP congestion control reacts by multiplying the current value of the congestion window with a number greater than 0 and less than 1.

However, since the increase in the window is so slow, the TCP connection may have to wait for many round-trip times before being able to efficiently use the available bandwidth. To avoid this, the TCP congestion control scheme includes the **slow-start** algorithm.

Quick Quiz!

1 Which of the following is not a requirement of a congestion control algorithm?

- A) Must avoid congestion
- B) Must be fair
- C) Must be efficient
- D) Must be accurate

COMPLETED 0%

1 of 2

Let's have a look at the slow start algorithm in the next lesson.

TCP Congestion Control: Slow Start

In this lesson, we're going to look at the slow start algorithm.

We'll cover the following

- How Slow Start Works
- Not All Congestion Is the Same!
  - Severe Congestion
    - Figure
  - Mild Congestion
    - Figure
- Quick Quiz!

How Slow Start Works

The objective of TCP slow-start is to quickly reach an acceptable value for the congestion window.

During slow-start:

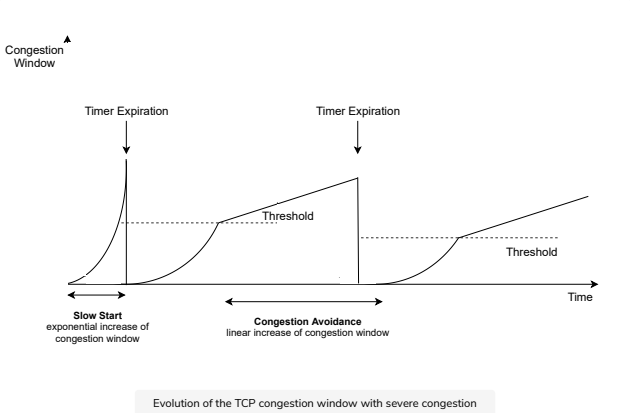
1. The congestion window is **doubled every round-trip time**.
2. The slow-start algorithm uses an additional variable in the TCB to maintain the **slow-start threshold**.
   - The slow-start threshold is an estimation of the last value of the congestion window that did *not* cause congestion.
   - It is initialized at the sending window and is updated after each congestion event.

3. The lost segments are retransmitted as the sender again performs slow-start until the congestion window reaches the slow start threshold.

4. It then switches to congestion avoidance and the congestion window increases linearly until segments are lost and the retransmission timer expires.

Figure

The figure below illustrates the evolution of the congestion window when there is severe congestion:

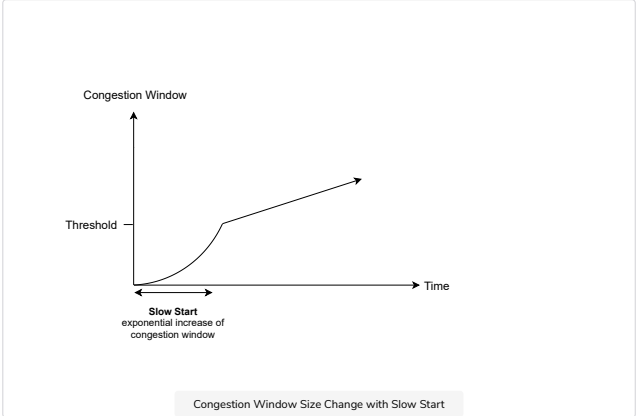


Evolution of the TCP congestion window with severe congestion

Mild Congestion

TCP considers that the network is lightly congested if it receives three duplicate acknowledgments.

1. The sender begins with a slow-start.
2. If 3 duplicate ACKs arrive, the sender performs a **fast retransmit** (retransmits without waiting for the retransmission timer to expire).
   - Have a look at the following slides to see when 3 duplicate acknowledgments could arrive and when a fast retransmit happens.



Congestion Window Size Change with Slow Start

Not All Congestion Is the Same!

The TCP congestion control scheme distinguishes between two types of congestion:

1. Severe Congestion
2. Mild Congestion

Let's discuss these one by one.

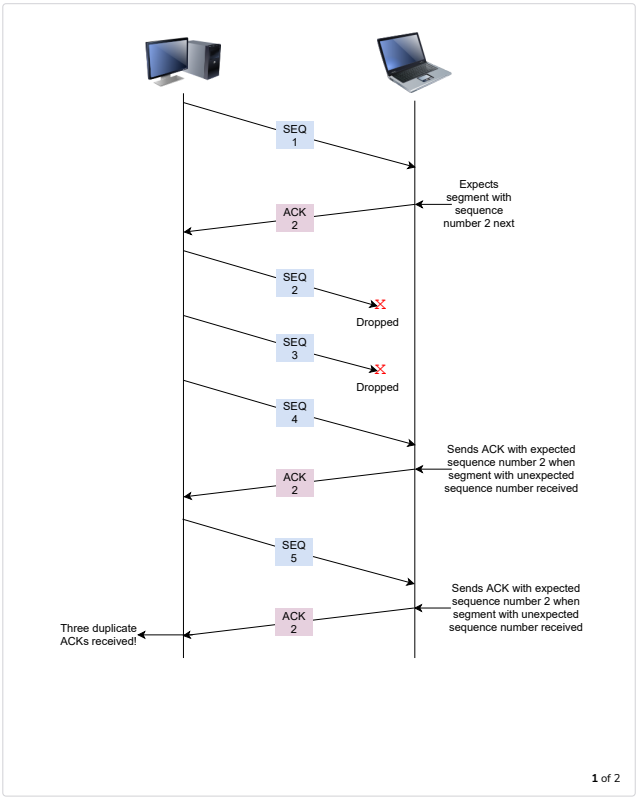
Severe Congestion

TCP considers that the network is severely congested when its retransmission timer expires. The following process is followed accordingly:

1. The sender performs slow-start until the first segments are lost and the retransmission timer expires.

2. At this time, TCP retransmits the first segment and the slow start threshold is set to half of the current congestion window. Then the congestion window is reset at one segment.

3. The lost segments are retransmitted as the sender again performs slow-



3. If the fast retransmit is successful, this implies that only one segment has been lost.

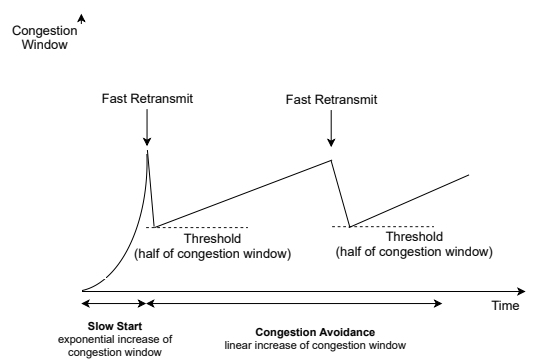
   - In this case, TCP performs multiplicative decrease and the congestion window is divided by 2.

- The slow-start threshold is set to the new value of the congestion window.

4. The sender immediately enters congestion avoidance as this was mild congestion.

Figure

The figure below illustrates the evolution of the congestion window when the network is lightly congested and all lost segments can be retransmitted using fast retransmit.



Evolution of the TCP congestion window when the network is lightly congested

Quick Quiz!

1 Slow start increases the congestion window size exponentially, whereas congestion avoidance increases the congestion window size linearly.

A) True

B) False

COMPLETED 0%

1 of 3

That's it for the transport layer! Let's look at socket programming in Python next!

The Basics

Congratulations, we made it! We can finally get to writing some code.

We'll cover the following

- Socket Programming
  - Types of Network Applications
- Socket Programming in Python

Socket Programming

- Recall that network applications usually consist of two programs: the **server** program and the **client** program. These programs reside on two separate end systems.
- When any of these programs want to communicate with another, they write the data they want to send to their **sockets**. The underlying protocols then deliver the data to the appropriate destination.

Types of Network Applications

- **Standard**: applications that use well-known protocols based on meticulous standards laid down by standards documents.
  - Pros: any other developer can write applications that are compatible with standard ones.
  - Cons: some customizability will be compromised.
- **Proprietary**: applications that use protocols of the developer's own design. The source code for such applications is generally not disclosed.
  - Pros: Extremely customizable, which allows for optimizing the application for particular use cases.
  - Cons: It's incredibly difficult to write applications compatible with a proprietary one. Also, designing protocols can lead to security loopholes.

🔎 **Did You Know?** Writing applications compatible with a proprietary one is doable but hard. A case in point is **Windows network file sharing**. Windows had a proprietary Server Message Block (SMB) protocol for file sharing. Only Windows clients could use Windows shared files, folders and printers. But then some hackers collected packet traces and reverse-engineered the protocol and came up with an open-source implementation named SAMBA, using which, Linux clients could access Windows file shares and even share files, folders, and printers with a Windows domain. A challenge with this approach is that when Windows updates their proprietary protocol, the open-source clients and servers don't talk to the new version of Windows and the hackers have to pull up their sleeves all over again.

Socket Programming in Python

Anyways, we're going to be learning the very basics of writing an application in Python. It's a great way to truly apply what you've learned.

Here's what to expect from the rest of this chapter:

1. We'll start with the very basics. You'll learn how to set up a UDP socket using the `socket` library.
2. We'll then write a complete server and a client program that runs on UDP. We'll also see them in action.
3. We'll then look at some improvements on the UDP server.
4. You'll get to do a project next! You'll be writing your very own chat app.
5. Lastly, we'll see how a few tweaks can change the program into one that runs on TCP.

Let's start with setting up a socket next!

Importing `socket`

The first step when writing a network application in Python is to import the `socket` library. It's generally already part of the Python bundle, so no extra library will have to be manually installed.

```
import socket
```

Creating a `socket` Object

We now create a **socket object** called `s` with a call to `socket.socket()`.

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print(s)
```

The syntax is as follows.

```
socket.socket(family, type, proto, fileno)
```

The syntax is shown here for completion, however, our main focus will be on explaining the **family** and **type** properties.

1. **Family**. The address family property is used to assign the type of addresses that a socket can communicate with. Only then can the addresses of that type be used with the socket. There are **three main options** available for this:
   - `AF_INET`. This family is used with **IPV4** addresses. When we first introduced IP addresses, we were talking about **IPV4** addresses. IP addresses are most commonly used.
   - `AF_INET6` Another address scheme, IPv6, was introduced since IPV4 is limited to about 4 billion addresses which are not sufficient, considering the exponential growth of the Internet. IPV6 provides 340 undecillion addresses (340×10^{36}). It's slowly being adopted. `AF_INET6` is used for IPV6 addresses. We'll introduce IPV6 more formally in a later lesson.
   - `AF_UNIX` This family is used for **Unix Domain Sockets (UDS)**, an interprocess communication endpoint for the same host. It's available on POSIX-compliant systems. Most operating systems today like Windows, Linux and Mac OS are POSIX compliant! So processes on a system can communicate with each other directly through this instead of having to communicate via the network.
2. **Type**. The type specifies the transport layer protocol:
   - `SOCK_DGRAM` specifies that the application is to use **User Datagram Protocol** (UDP). Recall that **UDP** is less reliable but requires no initial connection establishment. We are building these server and client programs pair in UDP.
   - `SOCK_STREAM` specifies that the application is to use **Transmission Control Protocol** (TCP). Recall that while TCP requires some initial setup, it's more reliable than UDP.

If you want to study the rest of the fields, have a look at the documentation. Default arguments are being used for the remaining arguments, which is fine in our example.

> **Note: INET socket vs UNIX socket** UNIX sockets are bound to a special file on your machine, whereas INET sockets are bound to an IP address and port. So UNIX sockets can only be accessed by the processes running on the machine and are therefore used for interprocess communication. INET sockets, on the other hand, can be accessed by remote machines.

Binding The Socket

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Setting up a UDP Socket

We're now going to write some basic server code in TCP. Let's get right into it.

We'll cover the following

- Purpose of the Program
- Importing `socket`
- Creating a `socket` Object
- Binding The Socket
  - Port Number
  - Hostname

Remember that sockets are just software endpoints that processes write and read data from. They are bound to an IP address and a port. As we will see, the sending process attaches the IP address and port number of the receiving application. The IP address and port number of the sending process are also attached to the packets as headers, but that's not done manually in the code of the application itself. Networking libraries are provided with nearly all programming languages and they take responsibility for lots of plumbing.

Purpose of the Program

Let's set up a socket for a **UDP server program** that works like so:

1. The client will send a line of text to the server.
2. The server will receive the data and convert each character to **uppercase**.
3. The server will send the uppercase characters to the client.
4. The client will receive and display them on its screen.

We are building the code line-by-line to make it easier to understand. Moreover, we will highlight new additions at every step.

```
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port))
```

Now, we bind the socket to an IP address and a port using the `bind()` function. It's given a certain n-tuple as an argument, where `n` depends on the family of the socket. In the case of `SOCK_DGRAM`, it's a tuple of the IP address and the port like the following.

```
socketObject.bind((IP address, port))
```

Port Number

We define the hostname and the port as variables on **lines 4 and 5**. The `port` is **3000** in this case, and as mentioned previously, the ports $0 - 1024$ should be avoided as they're reserved for other system-defined processes. Binding to them may generate an error as it may already be in use by another application.

Hostname

The hostname is the IP address that your server will listen on. You can set it to one of three options:

1. If you're following along on your local machine, set it to `127.0.0.1` which is the localhost address, for IPV4. This address is called the loopback or localhost address and you should use this because you'll be writing the client code on the same machine as the server. We're doing the same here.
2. You could also set it to the empty string `''` which represents the `INADDR_ANY`. This specifies that the program intends to receive packets sent to the specified port destined for any of the IP addresses configured on that machine.
3. Lastly, you could set it to a specific IP address assigned to your machine.

Now that we know how to set up a basic socket, which is necessary for any kind of network application using sockets, let's write some code that is specific to servers.

Writing a UDP Server

In the last lesson, we'd written code to setup an IPV4 socket on TCP. Let's now get into writing a program for a basic server.

We'll cover the following

- Setting a Purpose for the Server
- UDP IPV4 Socket
- Listening Infinitely
- Receiving Messages from Clients
- Capitalizing the Data
- Printing the Client's Message & Encoding
- Sending the Message Back to the Client

Setting a Purpose for the Server

In this lesson, we'll finish writing our server program. Our server will reply to every client's message with **a capitalized version of whatever a client program sends to it**. This is just what we're requiring of our server to do, there are other functions that such a server can perform as well.

So, in particular, the server will:

1. Print the original message received from the client.
2. Capitalize the message.
3. Send the capitalized version back to the client.

Let's code!

UDP IPV4 Socket

For reference, here is the code for a UDP socket.

```
import socket

# Setting up a socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port)) # Binding the socket to a port and IP address
print('Listening at {}'.format(s.getsockname())) # Printing the IP address and port of socket
```

You can use the `getsockname()` method on an object of the `socket` class to find the current IP address and port that a socket is bound to.

Listening Infinitely

Next, we set up a while loop (**lines 10 and 11**) so that the server listens infinitely. If the rest of this code weren't in this infinite while loop, the server would exit after dealing with one client.

```
import socket

# Setting up a socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port)) # Binding the socket to a port and IP address
print('Listening at {}'.format(s.getsockname())) # Printing the IP address and port of socket

while True:
    # The code to handle clients will go here
```

Receiving Messages from Clients

The server can now receive data from clients! The `recvfrom()` method (**line 12**) here accepts data of `MAX_SIZE_BYTES` length (declared on **line 3**) which is the size of one UDP datagram in bytes. This is to make sure that we receive the entirety of each packet. It also returns the IP address of the client that sent the data. We store the data and the client's IP address in the variables `data` and `clientAddress` respectively.

Note that the code stops and waits at `recvfrom()` until some data is received.

```
import socket

MAX_SIZE_BYTES = 65535 # Maximum size of a UDP datagram
```

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

# Setting up a socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port)) # Binding the socket to a port and IP address
print('Listening at {}'.format(s.getsockname())) # Printing the IP address and port of socket
while True:
    data, clientAddress = s.recvfrom(MAX_SIZE_BYTES) # Receive at most 65535 bytes at once
```

Capitalizing the Data

Next, we decode the message from the byte stream to ASCII (**lines 13 and 14**). Then we capitalize whatever the client sent.

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

# Setting up a socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port)) # Binding the socket to a port and IP address
print('Listening at {}'.format(s.getsockname())) # Printing the IP address and port of socket
while True:
    data, clientAddress = s.recvfrom(MAX_SIZE_BYTES)
    message = data.decode('ascii')
    upperCaseMessage = message.upper()
```

Printing the Client's Message & Encoding

We print the client's IP address next since it's always a good idea after a connection has been made in order to keep track of it (**line 15**). We also print the message they sent along with it.

We then encode the capitalized ASCII message to bytes (**line 16**).

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

# Setting up a socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port))
print('Listening at {}'.format(s.getsockname()))
while True:
    data, clientAddress = s.recvfrom(MAX_SIZE_BYTES)
    message = data.decode('ascii')
    upperCaseMessage = message.upper()
```

Writing a UDP Client Program

Let's now write a client to go with the server we wrote.

We'll cover the following

- The Server
- Creating a Client Socket
- Reading Data
- Sending It to the Server
- Receiving the Server's Response
- Decoding & Printing the Capitalized Message

The Server

Here's the server code that we have so far for reference.

```
1  import socket
2
3  MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram
4
5  # Setting up a socket
6  s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
7  port = 3000
8  hostname = '127.0.0.1'
9  s.bind((hostname, port))
10 print('Listening at {}'.format(s.getsockname()))
11 while True:
12     data, address = s.recvfrom(MAX_SIZE_BYTES)
13     message = data.decode('ascii')
14     upperCaseMessage = message.upper()
15     print('The client at {} says {!r}'.format(clientAddress, message))
16     data = upperCaseMessage.encode('ascii')
17     s.sendto(data, clientAddress)
```

Creating a Client Socket

Instead of explicitly binding the socket to a given port and IP as we did previously, we can let the OS take care of it. Remember ephemeral ports? Yes, the OS will bind the socket to a port dynamically. So all we really need is to create a UDP socket (**line 3**).

```
1  import socket
2
3  s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

In fact, we can check what address and port the OS assigned to the socket using the following line of code on **line 4**:

```
1  import socket
2
3  s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4  print('The OS assigned the address {} to me'.format(s.getsockname()))
```

You'll always get `(0.0.0.0,0)` for now because we haven't actually used the socket. You'll get the correct answer when we use the socket to send data in the next lesson.

Reading Data

Remember that the goal of this client and server was for the client to send a string to the server that it would capitalize and send back? Well, we'll get that string from the user's keyboard using the `Python3` function, `input()`. The function displays whatever prompt we specify, in this case the message 'Input lowercase sentence:' and waits for the user to provide an input from the keyboard (**line 4**). The user can then type a string of their choice and hit enter. The string will be stored in the variable `message`.

Next, we encode the ASCII encoded data to bytes using the `encode()` function (**line 5**).

```
import socket
```

```
    print('The client at {} says {!r}'.format(clientAddress, message))
    data = upperCaseMessage.encode('ascii')
```

Sending the Message Back to the Client

Lastly, we send the capitalized message back to the client using the `sendto()` function (**line 17**).

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

# Setting up a socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
port = 3000
hostname = '127.0.0.1'
s.bind((hostname, port))
print('Listening at {}'.format(s.getsockname()))
while True:
    data, clientAddress = s.recvfrom(MAX_SIZE_BYTES)
    message = data.decode('ascii')
    upperCaseMessage = message.upper()
    print('The client at {} says {!r}'.format(clientAddress, message))
    data = upperCaseMessage.encode('ascii')
    s.sendto(data, clientAddress)
```

Now we have a basic server that accepts messages from clients, has a defined purpose (capitalization), and responds to the client's messages. Let's write code for a client to go with this in the next lesson.

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input('Input lowercase sentence:' )
data = message.encode('ascii')
```

Sending It to the Server

We now send the message to the server using the `sendto()` function. In addition to the data, this function takes an IP address and a port number (**line 6**). We give it the IP address `127.0.0.1` and the port `3000` which we assigned to the server previously.

Also, notice that the `getsockname()` function will give us a useful answer at this point (**line 7**).

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input('Input lowercase sentence:' )
data = message.encode('ascii')
s.sendto(data, ('127.0.0.1', 3000))
print('The OS assigned the address {} to me'.format(s.getsockname()))
```

Receiving the Server's Response

We next receive the server's response and limit it to the size `MAX_SIZE_BYTES` which we saw previously.

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input('Input lowercase sentence:' )
data = message.encode('ascii')
s.sendto(data, ('127.0.0.1', 3000))
print('The OS assigned the address {} to me'.format(s.getsockname()))
data, address = s.recvfrom(MAX_SIZE_BYTES)
```

Decoding & Printing the Capitalized Message

Lastly, we decode and print the capitalized message that the server sent us.

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input('Input lowercase sentence:' )
data = message.encode('ascii')
s.sendto(data, ('127.0.0.1', 3000))
print('The OS assigned the address {} to me'.format(s.getsockname()))
data, address = s.recvfrom(MAX_SIZE_BYTES)
text = data.decode('ascii')
print('The server {} replied with {!r}'.format(address, text))
```

Now we have both our server and client programs. Let's see them live in action next!

Running The UDP Server & Client Together

We've spent the last few lessons writing code for a very basic client and a server. Let's see these in action in this lesson!

We'll cover the following

- Connecting the Two

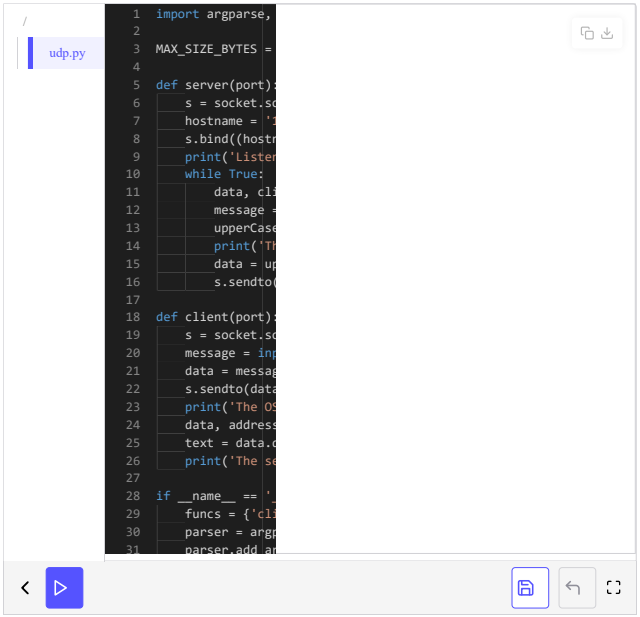
Connecting the Two

We'll run both together in one file called `udp.py` instead of running them separately. We've written some python code in the main function that allows you to specify which function you want the code to run, the `server` or the `client`.

To run the code, you would need to follow these steps:

1. Type your code and when you are ready to run the program, click on **Run**. The server code should start up automatically.
2. Open another terminal by clicking on +
3. Type the command `python3 /usercode/udp.py client` Note that it can be `server` in place of `client`.
4. Enter the text in the client window and see the effect.
5. If the program is not running to your satisfaction:
    1. Kill the running server program by typing the break sequence `ctrl+c` or `command+c` in both of the terminal windows.
    2. Change the code
    3. Click on **Run**.
    4. Type command `python3 /usercode/udp.py server` and `python3 /usercode/udp.py client` in the first and second terminal window, respectively. Go back to step 4.

> Every time you make a change to the code you must click **run** for the changes to take effect.



In the next lesson, we're going to look at some possible improvements to our current UDP program.

Improvements to UDP Programs: Avoiding Arbitrary Servers

There are a few improvements that can easily be made to our UDP program. Let's have a look.

We'll cover the following

- Problem: Replies From Arbitrary Servers
  - Fix with `connect()`
  - Disadvantages
  - Fix with Address Matching
- Quick Quiz!

Problem: Replies From Arbitrary Servers

Note that at the moment, our UDP client accepts replies from *any* machine and assumes that it's the one that it sent the initial message to, evident in the following line,

```
data, address = s.recvfrom(MAX_SIZE_BYTES)
```

Note how the client does not check **who** it is receiving the message from. It just receives a message.

Fix with `connect()`

There are two quick ways to go about fixing this. The first of which is to use the `connect()` method to forbid other addresses from sending packets to the client.

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

def client(port):
    host = 127.0.0.1
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect((host, port))
    message = input('Input lowercase sentence:' )
    data = message.encode('ascii')
    s.send(data)
    print('The OS assigned the address {} to me'.format(s.getsockname()))
    data = s.recv(MAX_SIZE_BYTES)
    text = data.decode('ascii')
    print('The server replied with {!r}'.format(text))
```

With the `sendto()` method, we had to specify the IP address and port of the server every time the client wanted to send a message. However, with the `connect()` method we used, we just use `send()` and `recv()` without passing any arguments about which address to send to because the program **knows** that.

This also means that no server other than the one the client *connected* to can send it messages. The operating system discards any of those messages by default.

Disadvantages

The main disadvantage of this method is that the **client can only be connected to one server at a time**. In most real life scenarios, singular applications connect to *multiple servers*!

Fix with Address Matching

A better, though more tedious approach, to handle multiple servers would be to **check the return address of each reply against a list of addresses that replies are expected from**.

Let's implement it!

```
import socket

MAX_SIZE_BYTES = 65535 # Mazimum size of a UDP datagram

def client(port):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    hosts = []
    while True:
        host = input('Input host address:' )
        hosts.append((host,port))
        message = input('Input message to send to server:' )
        data = message.encode('ascii')
        s.sendto(data, (host, port))
        print('The OS assigned the address {} to me'.format(s.getsockname()))
        data, address = s.recvfrom(MAX_SIZE_BYTES)
        text = data.decode('ascii')
        if(address in hosts):

            print('The server {} replied with {!r}'.format(address, text))
            hosts.remove(address)
        else:
            print('message {!r} from unexpected host {}!'.format(text, address))
```

As you can see, we created a list called `hosts` which contains tuples like $(IPaddresses,\ port\ numbers)$ of any host that the client connects to. Upon receiving every message, it checks whether the message is from a host it expects to receive a reply from. As soon as a reply is received, it removes the host from the list.

Quick Quiz!

1 Why do we remove the host's address from `hosts` once a reply is received?

- A) To evade attacks from arbitrary servers
- B) `hosts` only has addresses of hosts that we are waiting for a reply from, so the address is removed as soon as one is received
- C) To ensure that the replies are received in an orderly fashion

COMPLETED 0%

1 of 2

In the next lesson, you're going to try out an exercise for yourself: write a chat app in UDP!

Project: Write a UDP Chat App!

Welcome to your UDP project in this course!

We'll cover the following

- Instructions

Instructions

Writing a chat app is not so different from the capitalization code we saw in the last lesson. The idea is very simple. The client sends a message to the server and the server should respond with one. Both messages should be taken as input from the user. We've given you some basic starter code for it.

Here are some other factors you would want to consider to write your app:

1. Your client and server both need to stay alive and not exit after each message sent.
2. Both the client and the server need to print every message received from the other party.
3. The server should not be chatting with more than one client.

udp.py

```
import argparse,

MAX_SIZE_BYTES =

def server(port):
    pass
    # Your code g

def client(port):
    pass
    # Your code g

if __name__ == '_
    funcs = {'cli
    parser = argp
    parser.add_ar
    parser.add_ar
```

Solution: Write a UDP Chat App!

We'll cover the following

- Client
- Server

Client

The client program uses a `while` loop to keep the conversation with the server alive. Furthermore, it uses `connect()` to ensure that only one server is connected to, and only replies from that server are received.

```
import argparse, socket

MAX_SIZE_BYTES = 65535 # Mazim

def client(port):
    s = socket.socket(socket.A
    host = '127.0.0.1'
    while True:
        s.connect((host, port)
        message = input('Input
        data = message.encode(
        s.send(data)
        data = s.recv(MAX_SIZE_
        text = data.decode('as
        print('The server repl
```

Server

```
import argparse, socket

MAX_SIZE_BYTES = 65535 # Mazim

def server(port):
    s = socket.socket(socket.A
    hostname = '127.0.0.1'
    s.bind((hostname, port))
```

```
    args = parser
    function = fu
    function(arg
```

Note that to run the code, you would need to follow these steps:

1. Type your code and when you are ready to run the program, click on **Run**. The server code should start up automatically.
2. Open another terminal by clicking on +
3. Type the command `python3 /usercode/udp.py client` Note that it can be `server` in place of `client`.
4. Enter the text in the client window and see the effect.
5. If the program is not running to your satisfaction:
    1. Kill the running server program by typing the break sequence `ctrl+c` or `command+c` in both of the terminal windows.
    2. Change the code.
    3. Click on **Run**.
    4. Type command `python3 /usercode/udp.py server` and `python3 /usercode/udp.py client` in the first and second terminal window, respectively. Go back to step 4.

> Every time you make a change to the code you must click **run** for the changes to take effect

In the next lesson, we'll look at the solution to this project.

```
    print('Listening at {}'.fo
    while True:
        data, clientAddress = s
        message = data.decode(
        print('The client at {
        msg_to_send = input('I
        data = msg_to_send.enc
        s.sendto(data, clientA
```

Great! Let's look at how server and client programs can be written to run on TCP in Python3 in the next lesson!

A TCP Client-Server Program

In the last chapter, we studied TCP theory in detail. Now, we'll look at how we can code up TCP programs in Python.

We'll cover the following

- Introduction
- A TCP Server & Client Program
  - Handling Fragmentation
    - `sendall()`
    - `recvall()`

Introduction

There are a few key points to be noted about TCP programs:

- TCP `connect()` calls induce full TCP three-way handshakes! As we saw in the last chapter, **three-way handshakes can fail and so can `connect()` calls**.
- A **new socket gets created for every new connection** in a TCP architecture.
- **One socket** on TCP servers is dedicated to **continuously listen** for **new** incoming connections.
- When a connection is successful, that listening socket creates a **new socket exclusively for that connection**.
- When the **connection terminates**, the associated **socket** gets **deleted**.
- Every socket, and hence **each connection, is identified by the unique 4-tuple: `(local_ip, local_port, remote_ip, remote_port)`**. All incoming TCP packets are examined to see whether their source and destination addresses belong to any such currently connected sockets.

Handling Fragmentation

`sendall()`

One of three things may happen at every `send()` call:

1. All the data you passed to it gets sent immediately.
2. None of the data gets transmitted.
3. *Part* of the data gets transmitted.

The `send()` function returns the length of the number of bytes it successfully transmitted, which can be used to check if the entire segment was sent.

Here's what code to handle partial or no transmission would look like:

```
bytes_sent = 0 # No bytes initially sent
while bytes_sent < len(message): # If number of bytes sent is less than the amount of data
  message_left = message[bytes_sent:] # Indexing and storing the part of the message remainin
  bytes_sent += sock.send(message_left) # Sending remaining message
```

Luckily, Python has its own implementation of this in a function called `sendall()`. It ensures that all of the data gets sent. Check out line **37** in the TCP server-client program above to see how it is used.

`recvall()`

Unfortunately, no equivalent to automatically handle fragmentation exists for the receiving end. Hence, we'd have to cater for the cases when:

- Part of the sent data arrives
- None of the sent data arrives

We do this by defining a function called `rcvall()` on **line 3**.

That's the end of this chapter. In the next one, we'll start on the network layer.

- Unlike UDP, **TCP segments will be delivered as long as the sender and receiver are connected by a path and they are both live**.
- A sending TCP entity might **split TCP segments into packets** and so, receiving TCP entities would have to reassemble them. This is unlikely in our small program but happens all the time in the real world. So we need to take care of when there is data leftover in the buffer to send or to receive after one call.

A TCP Server & Client Program

tcp.py

```
1  import argparse, socket
2
3  def recvall(sock, length):
4      data = b''
5      while len(data) < length:
6          more = sock.recv(length - len(data))
7          if not more:
8              raise EOFError('was expecting %d bytes but only receive
9                             ' %d bytes before the socket closed'
10                            % (length, len(data)))
11         data += more
12     return data
13
14 def server(port):
15     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
16     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
17     sock.bind(('127.0.0.1', port))
18     sock.listen(1)
19     print('Listening at', sock.getsockname())
20     while True:
21         print('Waiting for a new connection')
22         sc, sockname = sock.accept()
23         print('Connection from', sockname)
24         print('  Socket name:', sc.getsockname())
25         print('  Socket peer:', sc.getpeername())
26         message = recvall(sc, 16)
27         print('  message from client:', repr(message))
28         sc.sendall(b'Goodbye, client!')
29         sc.close()
30         print('  Closing socket')
31
```

As you can see, the client program is pretty much the same as a UDP client program. There are a few key differences which we will explore now:

What Is the Network Layer?

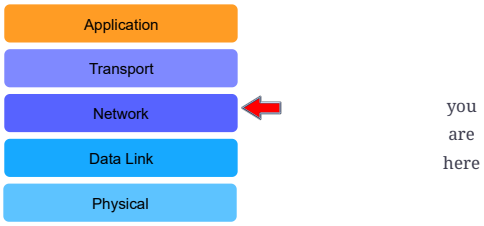
In this lesson, we'll get a quick introduction to the network layer.

We'll cover the following

- You Are Here!
- Main Objectives & Key Responsibilities
  - Limitations Of The Underlying Data Link Layer
  - Principles of the Network Layer
  - Network Layer Services
- Quick Quiz!

You Are Here!

Let's zoom out and have a look at the big picture.



Main Objectives & Key Responsibilities

The main objective of the network layer is to allow end systems to exchange information through intermediate systems called **routers**. The unit of information in the network layer is called a **packet**.

Limitations Of The Underlying Data Link Layer

Messages at the data link layer are called **frames**. There are more than a

dozen different types of data link layers.

1. Every data link layer technology has a limit on maximum frame size.
2. Most of them use a different maximum frame size.
3. Furthermore, each interface on an end system in the data link layer has a link layer address. This means the link layer has to have an addressing system of its own.

The network layer must cope with this heterogeneity of the data link layer.

Principles of the Network Layer

The network layer relies on the following principles:

1. Each network layer entity is identified by a **network layer address**. This address is independent of the data link layer addresses that the entity may use.
2. The service provided by the network layer **does not depend on** the service or the internal organization of **the underlying data link layers**. This independence ensures:
   - **Adaptability**. The network layer can be used by hosts attached to different types of data link layers.
   - **Independent Evolution**. The data link layers and the network layer evolve independently from each other.
   - **Forward Compatibility**. The network layer can be easily adapted to new data link layers when a new type is invented.
3. The network layer is conceptually divided into **two planes**:
   1. The **data plane**. The data plane contains the protocols and mechanisms that allow *hosts and routers to exchange packets carrying user data*.
   2. The **control plane**. The control plane contains the protocols and mechanisms that *enable routers to efficiently learn how to forward packets towards their final destination*.

Network Layer Services

There are two types of services that can be provided by the network layer:

- An unreliable connectionless service. This kind of service does not ensure message delivery and involves no established connections.
- A connection-oriented, reliable or unreliable, service. This kind of service establishes connections and may or may not ensure that messages are delivered.

Nowadays, most networks use an unreliable connectionless service at the network layer. This is our main focus in this chapter.

Quick Quiz!

1 What is the unit of information in the network layer called?

A) A segment

B) A datagram

C) A packet

COMPLETED 0%

1 of 3

In the next lesson, we'll look at the two most common ways that the network layer is organized.

Organization of the Network layer

We'll study the two internal organizations of the network layer in this lesson!

We'll cover the following

- Datagram Organization
  - Forwarding Tables
- Virtual Circuit Organization
- Virtual Circuit Organization vs. Datagram Organization
  - Advantages of Datagram Organization
  - Advantages of The Virtual Circuit Organization
- Quick Quiz!

There are two possible internal organizations of the network layer: **datagram** and **virtual circuits**.

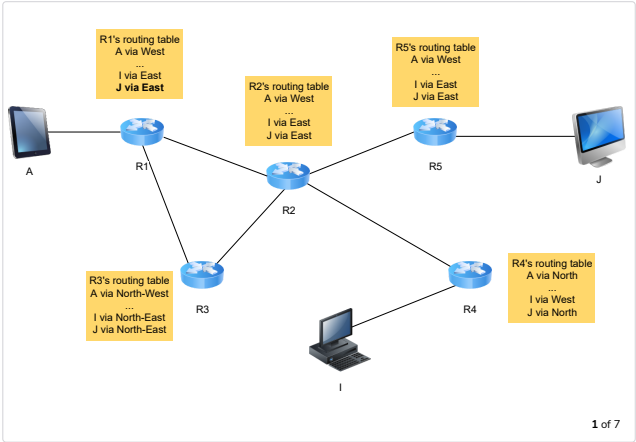
Datagram Organization

The datagram organization has been very popular in computer networks. Datagram-based network layers include **IPv4 and IPv6** in the **global Internet**, CLNP defined by the ISO, IPX defined by Novell or XNS defined by Xerox.

This organization is **connectionless** and hence each packet contains:

- The network layer address of the destination host.
- The network layer address of the sender.
- The information to be sent.

To understand the datagram organization, let's consider the slides below. A network layer address represented by a letter, has been assigned to each host and router. **Host A** wishes to send some information to **host J**.



Forwarding Tables

Routers use *hop-by-hop* forwarding in the datagram organization. This means that when a router receives a packet that is not destined to itself, it looks up the destination address of the packet in its **forwarding table**.

> A **forwarding table** is a data structure that maps each destination address to the device. Then, a packet must be forwarded for it to reach its final destination.

forwarding tables must

- Allow any host in the network to reach any other host. This implies that **each router must know a route towards each destination**.
- The paths composed from the information stored in the forwarding tables **must not contain loops**. Otherwise, some destinations would be unreachable.

unreachable.

The **data plane** contains all the protocols and algorithms that are used by hosts and routers to create and process the packets that contain user data.

The **control plane** contains all the protocols and mechanisms that are used to compute, install, and maintain forwarding tables on the routers.

> **Note: Routing Tables vs. Forwarding Tables** Routing tables are generally used to *generate* the information for a forwarding table, which is a subset of the routing table. So, a routing table may have 3 paths for one source, and destination pair generated from a few different algorithms that's perhaps also entered manually. The forwarding table, however, will only have one of those entries which is usually the preferred one based on another algorithm or criteria. The forwarding table is usually optimized for storage and lookup.

Virtual Circuit Organization

The second organization of the network layer, called **virtual circuits**, has been *inspired by the organization of telephone networks*.

- Telephone networks have been designed to carry phone calls that usually last a few minutes.
- Each phone is **identified by a telephone number** and is attached to a **telephone switch**.
- To initiate a phone call, a telephone first needs to send the destination's phone number to its local switch.
- The switch cooperates with the other switches in the network to create a bi-directional channel between the two telephones through the network.
- This channel will be used by the two telephones during the lifetime of the call and will be released at the end of the call.
- Until the 1960s, most of these channels were **created manually**, by telephone operators, upon request of the caller.
- Today's telephone networks use automated switches and allow several channels to be carried over **the same physical link**, but the principles remain roughly the same.

In a network using virtual circuits, all hosts are **identified with a network layer address**. However, a host must explicitly request the establishment of a virtual circuit before being able to send packets to a destination host. The request to establish a virtual circuit is processed by the control plane, which installs state to create the virtual circuit between the source and the destination through intermediate routers.

This organization is **connection-oriented** which means that resources like buffers, CPU, and bandwidth are reserved for every connection. The first packet sent reserves these resources for subsequent packets, which all follow a single path for the duration of the connection.

The virtual circuit organization has been mainly used in public networks, starting from X.25, and then Frame Relay and Asynchronous Transfer Mode (ATM) network.

Virtual Circuit Organization vs. Datagram Organization

Both the datagram and virtual circuit organizations have advantages and drawbacks.

Advantages of Datagram Organization

The main advantage of the datagram organization is that **hosts can easily send packets to any number of destinations**, while the virtual circuit organization requires the establishment of a virtual circuit before the transmission of a data packet. This can cause high overhead for hosts that exchange small amounts of data.

Another advantage of the datagram-based network layer is that **it's resilient**. If a virtual or physical circuit breaks, it has to go through the connection establishment phase, again. In case of datagram-based network layer, **each packet can be routed independently** of each other, hop-by-hop, so intermediate routers can divert around failures.

Advantages of The Virtual Circuit Organization

On the other hand, the main advantage of the virtual circuit organization is that the **forwarding algorithm used by routers is simpler** than when using the datagram organization. Furthermore, the utilization of virtual circuits may allow the **load to be better spread through the network**.

Also, since the packets follow a particular dedicated path, they **reach the destination in the order they were sent**. Virtual circuits can be configured to provide a variety of services including best effort, in which case *some* packets may be dropped. However, in case of bursty traffic, there is a possibility of packet drops.

Note that the Internet uses a **datagram organization**. We'll be focusing on that for the rest of the chapter.

Quick Quiz!

1 Which of the following is not a network layer principle?

A) The network layer is designed to work with specific flavors of the data link layer.

B) Each network layer entity is identified by a **network layer address**. This address is independent of the data link layer addresses that the entity may use.

C) The service provided by the network layer **does not depend on** the service or the internal organization of **the underlying data link layers**.

D) The network layer is conceptually divided into **two planes**.

COMPLETED 0%

1 of 3

In the next lesson, we'll look at how the control plane's routing algorithms work!

The Control Plane: Static & Dynamic Routing

In this section, we discuss the three main techniques that can be used to maintain the routing tables in a network.

We'll cover the following

- Modeling the Network as a Graph
- Static Routing
  - Disadvantages of Static Routing
- Dynamic Routing Algorithms
  - Distance Vector Routing
  - Link-State Routing
- Quick Quiz!

The main purpose of the *control plane* is to maintain and build routing tables. This is done via a number of algorithms and protocols which we will discuss here.

> **Note: Routing Algorithms vs. Routing Protocols**. The software/hardware *implementation* of routing algorithms are known as routing protocols!

Modeling the Network as a Graph

A network can be modeled as a **directed weighted graph**. Each router is a node, and the links between routers are the edges in the graph. A positive weight is associated with each directed edge. Routers use the shortest path to reach each destination. In practice, different types of weights can be associated with each directed edge:

- **Unit weight**. If all links have a unit weight, shortest path routing prefers the paths with the least number of intermediate routers.
- Weight **proportional to the propagation delay**. If all link weights are configured this way, shortest path routing algorithms will prefer the paths with the smallest propagation delay.
- Weight **proportional to the available bandwidth**. $weight = \frac{C}{bandwidth}$ where C is a constant larger than the highest link bandwidth in the network. If all link weights are configured this way, shortest path routing algorithms will prefer paths with higher bandwidth.

Usually, the same weight is assigned to the two edges that correspond to a physical link (i.e. R1→R2 and R2→R1). However, it's not necessary and some asymmetric links may have different bandwidths upstream and downstream.

Static Routing

Manually computed routes are manually added to the routing table. This is useful if there are a few outgoing links from your network. It gets difficult when you have rich connectivity (in terms of the number of links to other networks). It also does not automatically adapt to changes – addition or removal of links or routers.

Disadvantages of Static Routing

1. The main drawback of static routing is that it **doesn't adapt to the evolution of the network** and hence doesn't scale well. When a new route or link is added, all routing tables must be recomputed.
2. Furthermore, when a link or router fails, the routing tables must be updated as well.

Dynamic Routing Algorithms

Unlike static routing algorithms, dynamic ones adapt routing tables with changes in the network. There are two main classes of dynamic routing algorithms: **distance vector** and **link-state routing algorithms**.

Distance Vector Routing

Distance vector is a simple distributed routing protocol. Distance vector routing allows routers to discover the destinations reachable inside the network as well as the shortest path to reach each of these destinations. The shortest path is computed based on the **cost** that is associated with each link.

Link-State Routing

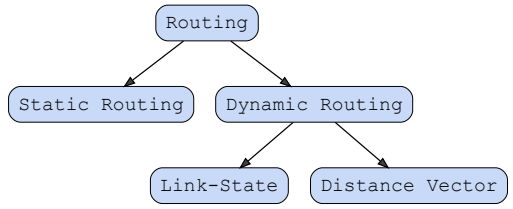
Another way to create a routing table with the most efficient path between two routers or 'nodes' is by using link-state routing.

Link-state routing works in two phases: **reliable flooding** and **route calculation**.

Routers running distance vector algorithms share summarized reachability information with their neighbors. Every router running link-state algorithms, on the other hand, builds a complete picture of the whole network (which is **phase I**) before computing the shortest path to all destinations.

Then, based on this learned topology, each router is able to compute its routing table by using the shortest path computation such as Dijkstra's Algorithm. This is **phase II**.

Here's a little roadmap of routing algorithm types.



Routing Algorithms

> **Note: Central Route Calculation** It's possible that in a network all the routes are computed centrally using a specific routing algorithm, and then downloaded to all the routers in the network. This approach can be used in relatively small networks but has scalability problems. This is usually achieved via architectures based on constructs such as Path Computation Element (PCE).
>
> On the other hand, routing tables can be computed in a distributed manner, whereby routers share reachability information with their neighbors. Then all routers independently run a routing algorithm to compute their local routing table. This is what we're mainly focusing on in this course.

Quick Quiz!

1 Given a network with 4 links with the following available bandwidth on each:

1. 100
2. 300
3. 40
4. 10

Which of the following is a possible value for C in the given equation to decide weights for each link?

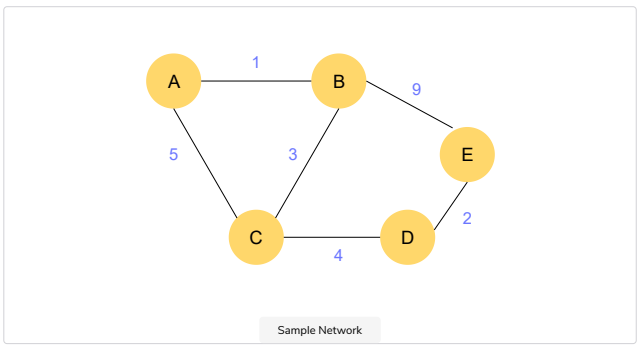
$weight = \frac{C}{bandwidth}$

- A) 10
- B) 40
- C) 300
- D) 301

COMPLETED 0%

1 of 3

Coming up next, we'll look at distance vector routing in-depth.



Sample Network

Initial Routing Table

What would the initial routing table look like at node **C** for example?

Destination	Cost	Next Hop
A	5	A
B	3	B
D	4	D

initial routing table at C

The table contains:

1. The names of the **destination nodes** which are the neighbors in this case.
2. The **initial cost** of the link to each of **C**'s neighbors,
3. The **'next hop' node**, i.e. the node that **C** would have to send a packet to in order for it to reach its destination. In this case, the next hop and the destination are the same since the destinations are all **C**'s neighbors.

The Control Plane: Distance Vector - Routing Information Protocol

In this lesson, we'll discuss the Routing Information Protocol, a popular distance-vector algorithm, based on the famous Bellman-Ford algorithm.

We'll cover the following

- Introduction
  - Initial State
    - Initial Routing Table
  - The Algorithm
    - Example
  - Count to Infinity Problem
  - Fix #1: Split Horizon
  - Fix #2: Split Horizon with Poison Reverse
- Quick Quiz!

Introduction

The **Routing Information Protocol (RIP)** based on the famous **Belmman-Ford** algorithm belongs to the distance vector class of routing algorithms and was used in ARPANET. While it used to be incredibly popular, it's not used very much now. There are other distance vector routing algorithms too such as Ford-Fulkerson.

Initial State

Each router or 'node,' maintains a **routing table** that initially contains the *estimated* cost to each of its neighbors.

Consider the following example of a small network where the yellow circles represent nodes, the black lines represent links, and the purple numbers represent the cost of each link.
Every node receives all of its neighbors' routing tables in two cases:

1. When a **trigger** happens such as a router or link failure happens.
2. Every N seconds, where N is a configurable parameter.

The Algorithm

Let's look at *how* the distance vector routing algorithm would arrive at the table above.

When a node **x** receives a routing table, `routing_table_y` from a neighbor **y**, the node applies the **Bellman Equation** in order to calculate/update the cost to reach each of **y**'s neighbors. Note that `routing_table_x` is the routing table of the node **x**.

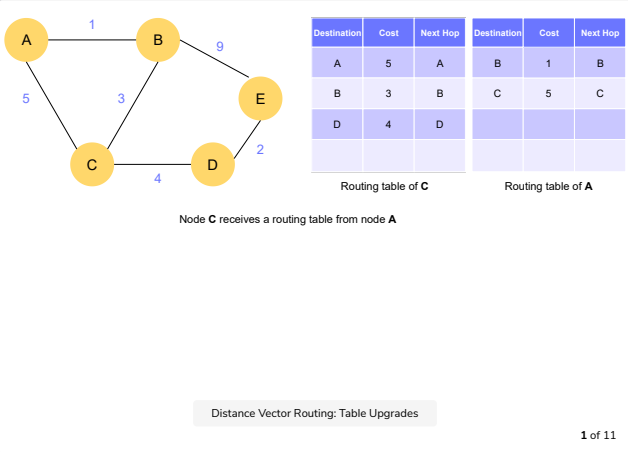
1. The Bellman Equation returns the **minimum** of the current estimated cost to reach a node, and the cost to reach a node *via* **y**.
2. If **x** does not have an estimated cost to reach a certain node, the cost to reach it is entered as the cost to reach the node *via* **y**.

Here's some pseudocode to clarify the picture.

```
for node in routing_table_y:
  if (node in routing_table_x):
    # Existing route, is the new one better?
     if (routing_table_x[node].cost > routing_table_x[y].cost + node.cost): # If current cost
       routing_table_x[node].cost = routing_table_x[y].cost + node.cost # Update
  else:
    # New route
    routing_table_x[node].cost = routing_table_x[y].cost + node.cost # Add entry
```

The estimated cost will finally converge to the optimal cost after a series of these message exchanges. Have a look at the following slides for an example:

Example



That being said, this algorithm has some issues.

Count to Infinity Problem

Consider the following scenario based on the given graph below:

1. Suppose the link between **D** and **E** fails.
2. **D** corrects its routing table such that the cost to reach **E** is infinity ∞.
3. The network will be triggered to exchange routing tables.
4. Suppose **C** gets a chance to advertise its routing table first and sends it over to **D**.
5. Since **C** does not yet know about the link failure between **D** and **E**, its routing table has an entry for a route to **E** with the cost of 6 via **D**.
6. **D** will notice that **C** has a route to **E** and will update its routing table with a route to **E** via **C**. The cost of the route will be the sum of the cost of **D** to **C**, and the cost C has to reach **E**: $4 + 6 = 9$.
7. When **C** receives **D**'s routing table, it will notice that **D** has changed its cost to reach **E** from 2 to 9 and will update its table accordingly to $4 + 9 = 13$. Then it advertises it to other neighbors.
8. Things will eventually converge at C with a cost of 12 to E and D will learn that it can reach E at a cost of 16.
9. So, the 'infinity', in this case is not very high. The problem is that this convergence to takes quite a while, and until the routers converge, there is a forwarding loop. Packets at C destined to E would go around in circles until their TTL expires.

There are a few improvements we can look at:

Fix #1: Split Horizon

This count to infinity problem occurs because a node **C** advertises to its neighbor **B** a route that it's learned from the neighbor **B** itself.

A possible solution to avoid this problem could be to change how a router creates its routing table. Instead of computing one routing table and sending it to all its neighbors, a router could create a routing table that's specific to each neighbor and **only contains the routes that have not been learned via this neighbor**. This technique is called **Split Horizon**.

Fix #2: Split Horizon with Poison Reverse

Another variant of this is called **Split-Horizon with Poison Reverse**. Nodes using this variant advertise a cost of ∞ for the destinations that they reach via the node to which they send their routing tables. Split Horizon with poison reverse converges faster than regular split horizon.

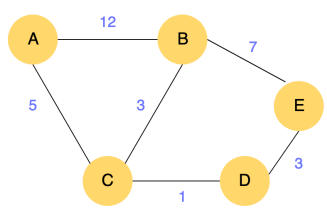
Quick Quiz!

Q What would the initial routing table look like at B for the following example with distance vector routing?

A	5	A
C	3	B
E	4	D

COMPLETED 0% 1 of 1

The most popular routing algorithms today use **link state routing**, which we'll look at in the next lesson.



A)

Destination	Count	Next Hop
A	5	A
B	3	B
D	4	D

B)

Destination	Count	Next Hop
A	12	A
C	3	C
E	7	E

C)

Destination	Count	Next Hop

Programming Challenge: Routing Information Protocol

In this lesson, you'll be writing code for the routing information protocol that we looked at previously.

We'll cover the following

- Problem Statement
- Starter Code
  - topology_reader.py
    - Sample Input
  - router.py
  - What Our Test Does?
  - port.py
  - rip_packet.py
- Coding Exercise

Problem Statement

In this challenge, you will implement the routing information protocol that we just studied in the last lesson! You're given some starter code files.

Starter Code

For this coding challenge, we are providing you with a network simulator written in `python3`. The implementation of our simplified version of RIP is also required in Python. Let's look at the starter code module by module.

`topology_reader.py`

This is the entry point to our code. It takes a network topology in the form of a Python list as input and returns a list of `router` objects that reflect that topology. Here's what the topology looks like:

- The `router` class **inherits** the `router_base` class and is the class **you'll be working in**. In particular, you'll be writing the functions

  `send_RIP_packets()` and `receive_RIP_packets()`.

What Our Test Does?

Our testing code is simply a **network simulator** that supplies a number of network topologies, creates a list of routers with `topology_reader()` and calls `send_RIP_packets()` on each router `steps_to_run` number of times where `steps_to_run` is randomized. It then stores the list of routers returned from the `send_RIP_packets()` function and checks if they are as expected. Note that our testing code is not visible to you.

`port.py`

This file contains the classes `port_link` and `port`.

- The `port_link` class defines the links on each port. This class consists of the destination router's IP address (`dest_IP_address`), the destination router's port's IP addresses (`dest_port_IP`) and the cost of the link (`cost`).
- The `port` class has two attributes: the IP address of the port (`port_IP`), and the link on the port (`link`) which is an object of the class `port_link`.

The `topology_to_routers` function from the `topology_reader.py` file first creates a links, then ports and then finally passes them to router objects upon creation.

`rip_packet.py`

This file consists of two classes: `RIP_entry` and `RIP_packet`.

- `RIP_entry`: each object of this class represents an entry of the forwarding table of the router. Each object of the `router` class consists of a list of these which makes up its forwarding table. Each `RIP_entry` object will have the following attributes: The IP address of the sending port (`port_IP`), the cost of sending on this link (`cost`), the IP address of the destination router (`dest_IP_address`), and the IP address of the next hop router (`next_hop_IP`).
- `RIP_packet`: when a router wants to forward its RIP entries, it does so by creating an object of this class. This class consists of the list of RIP entries

Sample Input

```
topology = [
  [1, [11, 2, 21, 1], [12, 4, 41, 1]],  # Routers and ports
  [2,[21, 1, 11, 1],[22, 5, 53, 1], [23,3,31,1]], #[IP of router, [port of router, IP of
  [3,[31,2,23,1],[32,5,52,1]],
  [4,[41,1,12, 1],[42,5,51,1]],
  [5,[51,4,42,1],[52,3,32,1],[53,2,22,1]]
]
```

The list consists of sublists. Each sublist represents **one** router. So `[1, [11, 2, 21, 1], [12, 4, 41, 1]]`, for instance, represents a router.

- The first element of this sublist is the IP address of the router. It is `1` in the case of the example above.
- The rest of the elements of this sublist are other lists that represent ports and their links to ports on other routers.
- There can be as many of these sub-sublists as there are ports.
- Each port-link sub-sublist is as follows:
  - `[IP of port of router this router, IP of destination router, IP of port of destination router, cost of link]`
- So the topology looks like:
  - `[[IP of router, [IP of port of this router, IP of destination router, IP of port of destination router, cost of link], [IP of router, [IP of port of this router, IP of destination router, IP of port of destination router, cost of link], ...]`
- Note that a link between two routers **has** to be present in both. So a link to a port on a router with IP `2` from a router with IP `1` `[1, [11, 2, 21, 1] , [12, 4, 41, 1]]` is reflected in the sublist of router with IP `2`, as follows: `[2, [21, 1, 11, 1] ,[22, 5, 53, 1], [23,3,31,1]]`

`router.py`

This file contains two classes: `router_base` and `router`.

- The `router_base` class contains the IP address, a list of RIP entries and a list of ports for each router, along with some functions that will help you implement the protocol. The IP address is self-explanatory but we'll get to the other two in a minute.

  a few useful methods.

Coding Exercise

Great! Now you have some background on the code. Note that we haven't discussed the skeleton code in its entirety so you should read it to understand the methods provided. Try the challenge yourself in the widget below!

> Note that `main.py` is empty. That's okay, don't worry about it.

main.py
port.py
rip_packet.py
router.py
topology_reader.py

```
from rip_packet import RIP_entry
from rip_packet import RIP_packet
from port import port
from port import port_link

class router_base:
  def __init__(self, IP_address, rip_entries, ports):
    self.IP_address = IP_address
    self.rip_entries = rip_entries
    self.ports = ports

  def add_port(self, prt):
    self.ports.append(prt)

  def add_RIP_entry(self, port_IP, dest_IP, cost, next_hop_IP):
    new_rip_entry = RIP_entry(port_IP, cost, dest_IP, next_hop_IP)
    self.rip_entries.append(new_rip_entry)

  def find_RIP_entry(self, destination_IP_to_find):
    for entry in self.rip_entries:
      if(entry.dest_IP_address == destination_IP_to_find):
        return entry
    return None

  def set_RIP_entry_cost(self, destination_IP_to_find, new_cost):
    for i in self.rip_entries:
      if(entry.dest_IP_address == destination_IP_to_find):
        entry.set_cost(new_cost)
    return None

  def delete_RIP_entry(self, destination_IP_to_find):
```

```
    for entry in self.rip_entries:
      if(entry.dest_IP_address == destination_IP_to_find):
        self.rip_entries.remove(entry)


  def print_router(self):
    print("~~~~ Router IP address = " + str(self.IP_address) + "~~~~")
    print("---Ports---")
    print("Port IP | Destination Router IP | Destination Port IP | Cost")
    for p in self.ports:
      p.print_port()
    print("---RIP entries---")
    print("port IP | destination IP address | next hop | cost")
    for re in self.rip_entries:
      re.print_rip_entry()

  def return_router(self):
    r =[]
    r.append("~~~~ Router IP address = " + str(self.IP_address) + "~~~~")
    r.append("---Ports---")
    r.append("Port IP | Destination Router IP | Destination Port IP | Cost")
    for p in self.ports:
      r.append(p.return_port())
    r.append("---RIP entries---")
    r.append("port IP | destination IP address | next hop | cost")
    for re in self.rip_entries:
      r.append(re.return_rip_entry())
    return r

class router(router_base):
  def send_RIP_packets(self, routers):
    # Write your code here
    return routers

  def receive_RIP_packets(self, rip_packet, link_send_on, routers, next_hop_IP):
    # Write your code here
    return routers
```

In the next lesson, we’ll look at a detailed analysis of the exercise.

Explanation

`send_RIP_packets()`

Let’s go through the solution line-by-line:

- **line 62**: We create the RIP packet that we’ll send to all of our neighbors on this line. We pass the `rip_entries` list and the length of that list on this line.
- **lines 63-66**: we now find the neighbors of this router. We do this by iterating over the given list of routers in the network and checking to see if any of our ports have a link to them. We do this by iterating over our ports and checking each port’s link’s destination IP address against the router’s IP address. If they match, the router is a neighbor and we send it our RIP packet by calling `receive_RIP_packets()` on it.

Finally, the routers list is returned.

`receive_RIP_packets()`

This function consists of the core of the Routing Information Protocol. It works as follows:

1. The receiving router checks for all RIP entries in the received RIP packet if they exist in its own RIP entries list.
2. If an entry is for a destination that’s **not found** in the receiving router’s RIP entries list, it adds it as done on **lines 85-88**.
3. If an entry is for a destination that **is found** in the receiving router’s RIP entries list, it does one of two things:
    1. If the entry is from a router whose IP address **is equal** to the next hop IP in the router’s current RIP entry **and** the cost has changed, it simply sets the cost as the minimum of 16 and the new cost. Note that the new cost can be greater or lesser than the current cost.
    2. Otherwise, if the cost advertised in this RIP entry is lesser than the one the router currently has, it sets its RIP entry to the one through the router that sent the RIP entry.

Finally, the routers list is returned.

Solution Review: Routing Information Protocol

In this lesson, we'll analyze the solution to the RIP challenge.

We'll cover the following

- Solution
- Explanation
    - `send_RIP_packets()`
    - `receive_RIP_packets()`

Solution

main.py

port.py

rip_packet.py

router.py

topology_reader.py

The Control Plane: Link State Routing

In this lesson, we'll study link state routing.

We'll cover the following

- Phase I: Reliable Flooding
  - Neighbor Discovery
  - LSPs
  - Flooding Algorithm
  - Handling Link Failure
    - Two-way Connectivity Check
    - Handling Router Failure
- Quick Quiz!

-Another way to create a routing table with the most efficient path between two routers or 'nodes' is by using **link-state** routing.

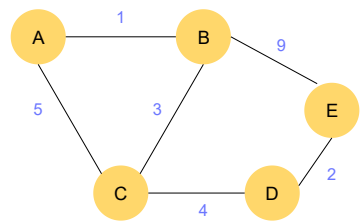
Link state routing works in two phases: **reliable flooding** and **route calculation**. Let's look at phase I now.

Phase I: Reliable Flooding

Neighbor Discovery

When a link-state router boots, it first needs to discover its neighbors. To do this:

- Each router sends a **HELLO** message every N seconds on all of its interfaces. This message contains the router's unique address.
- As its neighboring routers also send **HELLO** messages, the router discovers its neighbors.
- These **HELLO** messages are only sent to neighbors that are **directly** connected to a router, and a router never forwards the **HELLO** messages that they receive.
- **HELLO** messages are also used to detect link and router failures. A link is considered to have failed if no **HELLO** message has been received from the neighboring router for a period of $k \times N$ seconds.



1 of 6

LSPs

Once a router has discovered its neighbours, it must reliably distribute its local links to all routers in the network to allow them to compute their local view of the network topology. For this, each router builds a link-state packet (LSP) that contains the following information:

- `LSP.Router`: identification (address) of the sender of the LSP
- `LSP.age`: age or remaining lifetime of the LSP
- `LSP.seq`: sequence number of the LSP
- `LSP.Links[]`: links advertised in the LSP. Each directed link is represented with the following information:
  - `LSP.Links[i].Id`: identification of the neighbour
  - `LSP.Links[i].cost`: cost of the link

Flooding Algorithm

The routers will construct their routing tables based on the LSPs.

The **Flooding Algorithm** is used to efficiently distribute the LSPs of all routers. Each router that implements flooding maintains a **link state database (LSDB)** that contains the most recent LSP sent by each router. When a router receives an LSP, it:

- Verifies whether this LSP is already stored inside its LSDB. If so, the router has already distributed the LSP earlier and it **does not need to forward it**.
- Otherwise, **the router forwards the LSP on all links** except the link over which the LSP was received.

To ensure that all routers receive all LSPs, even when there are transmission errors, link state routing protocols use **reliable flooding**, which involves acknowledgments, and if necessary, retransmissions to ensure that all link state packets are successfully transferred to all neighbouring routers.

> **What We Have so Far:** Routers combine the received LSPs with their own LSP to compute the entire network topology.

Then in phase II, the routers apply shortest path algorithms to compute the most efficient path.

Handling Link Failure

- When a link fails, the two routers attached to the link **detect the failure by the lack of HELLO messages** received in the last k×N seconds.
- Once a router has detected a local link failure, it generates and **floods a new LSP that no longer contains the failed link**, and the new LSP replaces the previous LSP in the network.
- As the two routers attached to a link do not detect this failure exactly at the same time, the status of the link may be advertised differently by one of the routers.

Two-way Connectivity Check

- When a link is reported in the LSP of only **one** of the attached routers, the rest consider the link to have failed in **both directions** and remove it from the directed graph that they compute from their LSDB! This check allows link failures to be flooded quickly, as a single LSP is sufficient to announce such bad news.
- Furthermore, a link can only be used once the two attached routers have sent their LSPs.

Handling Router Failure

- The two-way connectivity check also allows for dealing with **router failures**. When a router fails, all its links fail by definition. Unfortunately, it doesn't send a new LSP to announce its failure. The two-way connectivity check ensures that the failed router is removed from the graph.
- When a router fails, its LSP must be removed from the LSDB of all routers. This can be done by using the **age field** that is included in each LSP. When a router generates an LSP, it sets the `LSP.age` to a value called the LSP's **lifetime** (usually measured in seconds). All routers regularly decrement the age of the LSPs in their LSDBs and an LSP is discarded once its age reaches 0.

Quick Quiz!

1 Why are routing loops uncommon in networks that use link state routing?

A) Each router builds a partial view of the network based on number of nodes

B) Routers flood the network and use LSAs to discover any loops.

C) Each router builds a comprehensive and updated view of the network.

COMPLETED 0%

1 of 2

In the next lesson, we'll study Dijkstra's algorithm!

The Control Plane: Route Calculation - Dijkstra's

In this lesson, we'll study Dijkstra's shortest path algorithm!

We'll cover the following

- Phase II: Route Calculation
- Dijkstra's Algorithm
  - Algorithm
  - Finding the Shortest Path
- Visual Example
- Quick Quiz!

Phase II: Route Calculation

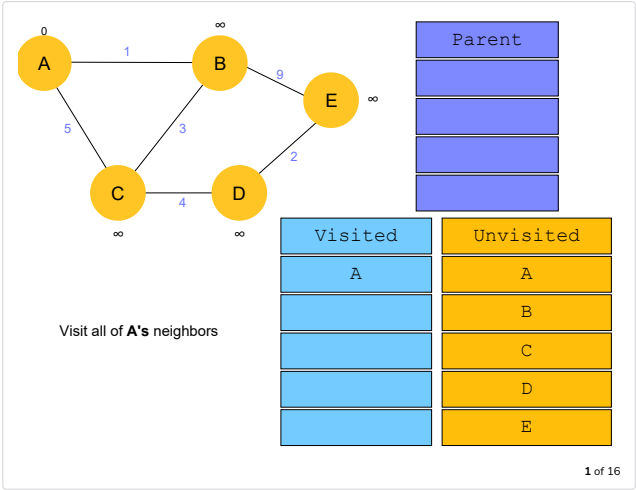
Each router then computes the spanning tree rooted at itself and calculates the entries in the routing table by using **Dijkstra's shortest path algorithm**. Dijkstra's is a common algorithm that is usually taught in *Algorithms* or *Data Structures* classes. Let's get a quick refresher of it.

Dijkstra's Algorithm

The goal is to find the shortest path from an **initial node** to all other nodes in the graph.

We first need to set up some data structures for us to use throughout the algorithm.

1. Create a set called the **unvisited set**. All the nodes are initially unvisited.
2. Create a set called the **visited set**. It's initially empty.
3. Create a list called the **parent** list. It will contain mappings of nodes to their parents.
4. Lastly, every node has a distance of it from the initial node. Initially, all the nodes besides the initial node itself have a starting distance of infinity. We call this `d_node_n`,
5. Every link between two nodes in the graph has a certain weight. We call this `w_node_n_node_m`.



Algorithm

1. Start with the **initial node** in the graph. Mark it as the **current node**.
2. Consider each of its neighbor's that are NOT in the **visited** set.
3. If the sum of the distance of the current node and the distance to the neighbor from the current node is **lower** than the current distance of the neighbor, replace it with the new distance.
   - In other words, if `w_node_curr_node_n` + `d_node_curr` < `d_node_n`, set `d_node_n` to `w_node_curr_node_n` + `d_node_curr`.
   - Also, set the parent of this neighbor, `n`, to the current node.
4. Repeat step 3 for all unvisited neighbors. After that, add the current node to the visited set.
5. Repeat steps 2-4 for the neighbor with the lowest `d_node_n`. Continue until the entire graph is visited.

Finding the Shortest Path

To find the shortest path from a given node **n** to the initial node:

1. Find the parent of the current node. Initially the current node is **n**.
2. Set the current node to the new parent node.
3. Store each 'current node' in a stack.
4. Repeat steps 1-3 until the initial node is reached.
5. Pop and print the contents of the stack until it is empty.

Visual Example

Have a look at the following example to see how Dijkstra's would apply to a graph.

Quick Quiz!

1 What is the aim of Dijkstra's Algorithm?

A) To find the shortest paths from every node to every other node in a given graph

B) To find the most efficient paths from every node to every other node in a given graph

C) To find the most efficient paths from a node in a given

graph to every other node in the graph

COMPLETED 0%

1 of 3

In the next lesson, you'll implement Dijkstra's Algorithm!

Programming Challenge: Implementing Dijkstra's

In this lesson, you'll be implementing Dijkstra's Shortest Path Algorithm.

We'll cover the following

- Problem Statement
  - Input
  - Output
  - Sample Input
  - Sample Output
- Coding Exercise

Problem Statement

Given an adjacency matrix in a 2D array, solve the **Single Source Shortest Path** algorithm, essentially by implementing the Dijkstra's algorithm discussed in the previous lesson. We've written some skeleton code for the function.

1. The value of the weight of the link is `graph[src][dst]`.
2. The graph is undirected so `graph[src][dst]==graph[dst][src]`.
3. A link between the `src` and `dst` exists if `-1<graph[src][dst]<16`.
4. If `graph[src][dst]>=16` the weight of the link is infinite and it does not function.

Input

1. An adjacency matrix, i.e., a 2D array, a source node, and a destination node.

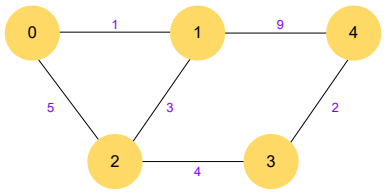
Output

The **shortest path** between the source and destination in the form of an array of integers where each integer represents a node and the **total weight** of the path.

Sample Input

1.
```
graph = [
  [0,1,5,-1,-1],
  [1,0,3,-1,9],
  [5,3,0,4,-1],
  [-1,-1,4,0,2],
  [-1,9,-1,2,0]
]
```

This adjacency matrix represents the following graph:



2. A source and destination:

```
src = 0
dst = 3
```

Sample Output

```
shortest_path = [0,1,2,3]
cost = 8
```

Coding Exercise

Try it yourself below!

```
def Dijkstra(graph, src, dst):
  pass
```

In the next lesson, we'll look at a solution to this problem.